MMA3001 Numerical Methods and Machine Learning: Individual Project Report

Predicting Battery Electric Vehicle Energy Consumption from EPA Test Data: Physics, Machine Learning and Hybrid Models

Harshil Dobariya | Student ID 34978232 | Monash University, Semester 2 2026

GitHub: https://github.com/Harshil-Dobariya/MMA3001_PROJECT_HARSHILDOBARIYA-EV_ENERGY_PREDICTION

Summary

This project asks a simple question: can you predict how much energy a battery electric vehicle (EV) will use from the numbers known early in its design? I used 712 EVs from the US EPA's test data (model years 2022 to 2026) and predicted their unadjusted energy consumption, in kWh/100 mi, on the EPA city (UDDS) and highway (HWFET) cycles. I built a physics model that integrates road-load and inertia power over the measured speed traces, then compared it with linear regression, a decision tree, support-vector regression (SVR), a neural network, and four hybrids in which an ML model learns whatever the physics gets wrong.

The result surprised me. Cross-validation picked the physics + SVR hybrid, but on 53 vehicle models it had never seen, its city test RMSE was 3.82 kWh/100 mi against 3.62 for physics alone. It extrapolated badly. The three-parameter physics model ended up being the one I would trust: it was the most stable across folds, the best on new 2025 to 2026 models (highway RMSE 1.18 kWh/100 mi, R^2 0.84), and it responded the right way to mass and drag for every single vehicle.

1. Engineering problem

An EV's driving range is roughly its usable battery energy divided by how much energy it uses per mile. The battery size is mostly a product and cost decision. Consumption is where the engineering happens, because it depends on the car's mass, its aerodynamic drag, its tyres, and how efficient the drivetrain and regenerative braking are. Cutting consumption by 10% buys the same extra range as a battery about 11% bigger, without carrying the extra weight or paying for the cells.

So the problem I set out to solve was predicting a vehicle's consumption on the two standard EPA laboratory cycles from parameters that are known early in a design: test mass, rated power, drivetrain layout, and the three road-load coefficients that describe how much force it takes to push the car along at each speed. A good prediction would let an engineer ask what 200 kg less mass, or a slipperier body, would be worth before anyone builds a prototype. It is meant for early estimates and comparing design options, not to replace certification testing. I kept the scope deliberately narrow in four ways:

- It only covers the two laboratory cycles driven on a dynamometer, so real-world driving, very hot or cold weather and air-conditioning use are outside it.
- It predicts unadjusted (laboratory) consumption. Window-sticker figures can be estimated from it with EPA's approximate 0.7 adjustment, which I do in Section 4.6.
- It can only use what EPA records. Motor type, tyre specification, battery chemistry and thermal management are not in the data.
- It predicts consumption, not range. I originally proposed a range prediction, but the EPA test data has measured consumption and no range or battery capacity, so I changed the target with Callum's approval. I recover range for two cars with published battery capacities in Section 4.6.

2. Inputs, outputs and data

2.1 Data sources

The vehicle data is the US EPA's *Data on Cars Used for Testing Fuel Economy* for model years 2022 to 2026 (US Environmental Protection Agency, n.d.-a). It comes as five Excel .xlsx spreadsheets, one per model year, with 21,479 rows between them. Each row is one laboratory test of one vehicle configuration, with mass in lb, road-load coefficients in lbf-based units, and efficiency in either MPGe or kWh/100 mi. The UDDS (1,369 s, 7.45 mi) and HWFET (765 s, 10.26 mi) speed traces are EPA's published dynamometer drive schedules (US Environmental

Protection Agency, n.d.-b), which are tab-separated text files of time in seconds and speed in mph at 1 s resolution. Both are public US Government data, so licensing and privacy were not an issue.

2.2 Inputs and outputs

Table 1. Top-level inputs, as used by the models after conversion to SI units.

Input	Unit	Range in data	Source and notes
Equivalent test mass, m	kg	1,474 to 4,082	Converted from lb; the inertia the dynamometer simulates
Rated power	hp	98 to 1,839	As reported to EPA
Road-load coefficient f_0	N	74 to 295	From EPA target coefficient A (lbf); mostly rolling resistance
Road-load coefficient f_1	N/(m/s)	−2.6 to 13.7	From B (lbf/mph); speed-dependent losses
Road-load coefficient f_2	N/(m/s)²	0.22 to 0.82	From C (lbf/mph²); mostly aerodynamic drag
Drivetrain	category	5 types	e.g. All Wheel Drive, 2-Wheel Drive Rear
Vehicle type	category	Car, Truck, Both	EPA classification
Drive-cycle speed, v(t)	m/s	0 to 26.8	1 s steps; converted from mph

The road-load force, $F = A + Bv + Cv^2$, comes from coast-down tests on a real road. It already lumps together the aerodynamic, tyre and some driveline losses for each specific car.

Table 2. Top-level outputs.

Output	Unit	Meaning and intended use
City consumption	kWh/100 mi	Predicted AC energy drawn from the wall per 100 miles on the UDDS, unadjusted
Highway consumption	kWh/100 mi	The same on the HWFET
Label MPGe (derived)	MPGe	Approximate window-sticker value, 0.7 × 3,370.5 / consumption
Label range (derived)	miles	Approximate EPA range from a known battery capacity and an assumed charging efficiency

2.3 Data cleaning and handling of invalid inputs

Cleaning took the 21,479 raw rows down to 712 vehicles (263 vehicle models) with both a city and a highway result. The count after each step is in Appendix Table A4. None of the three problems below would have raised an error. The code would just have given wrong answers.

- The efficiency column mixes units. EPA labels it "MPG", and most manufacturers report unadjusted MPGe (roughly 70 to 200), but Hyundai, Kia, Genesis, Rivian and a few others report kWh/100 mi (roughly 15 to 40). I infer the unit from the size of the number, using 40 as the threshold, and convert MPGe with kWh/100 mi = 100 × 33.705 / MPGe, where 33.705 kWh is EPA's energy equivalent of a gallon.
- Some values are window-sticker figures, not test results. When I ran the physics model, 28 vehicles reported as whole-number MPGe on both cycles sat about 1.3 times above prediction, which is what you would expect from label values (about 0.7 times the raw result). I checked this myself on fueleconomy.gov. Three 2024 GMC Hummer EV SUV versions show 59/48, 56/46 and 51/43 MPGe city/highway in both places, and the 2022 Chevrolet Bolt EV label (131/109) is within 2 MPGe of the data. I excluded these vehicles with a rule based on how the values were reported, not on how badly the model fitted them, because removing points the model disagrees with would be circular.
- Many vehicles were tested more than once. I average the repeats, and their median spread of 0.16 kWh/100 mi gives a measurement noise floor that no model can honestly beat. Carryover models retested in later years share a group label, so the train/test split can never put copies of the same car on both sides (Section 4.2).

Physically implausible results (outside 10 to 80 kWh/100 mi, or a negative road load at 60 mph) and placeholder values (999 MPGe, or power under 50 hp) are removed. A drivetrain category the models have not seen is encoded as zeros rather than crashing the pipeline, and any test vehicle with an input outside the 1st to 99th percentile of the training data is flagged, because that is where data-driven models start extrapolating (Section 4.4). The inputs and outputs of individual functions are documented in NumPy-style docstrings and in the HTML documentation generated with pdoc (pdoc, n.d.).

3. Computational approach

I wanted a fair fight between a model built from physics and models that learn purely from data. The physics model uses the numerical methods from Weeks 6 and 7 and has only three parameters, each with a physical meaning. The data-driven models are the Week 5 regression methods.

3.1 Physics model

On a flat dynamometer, Newton's second law along the direction of travel gives the tractive force the motors have to deliver at the wheels:

$$F_t(t) = m\,a(t) + f_0 + f_1\,v(t) + f_2\,v(t)^2 \tag{1}$$

where m is the equivalent test mass (kg), v and a are the speed (m/s) and acceleration (m/s²) from the cycle, and f_0, f_1 and f_2 are the road-load coefficients in SI units. Wheel power is $P_w(t) = F_t(t)\,v(t)$. It is positive when the motors are driving the car and negative when the car is slowing faster than road load alone would slow it, which is the energy regenerative braking can try to recover. I model the energy drawn from the wall over a cycle of length T as

$$E_b = \frac{E^+}{\eta_d} - \eta_r E^- + P_{\text{aux}}\,T, \qquad E^+ = \int_0^T \max(P_w, 0)\,\mathrm{d}t, \qquad E^- = \int_0^T \max(-P_w, 0)\,\mathrm{d}t \tag{2}$$

where η_d is the overall wall-to-wheel efficiency when driving (charger, battery, inverter, motor and gearbox together), η_r is the fraction of braking energy recovered, and P_{aux} is a constant accessory load in watts. Consumption is $c = E_b / D$, where $D = \int v\,dt$ is the distance, converted from J/m to kWh/100 mi by multiplying by $160{,}934.4 / (3.6 \times 10^6)$.

The useful part of this form is that it is linear in $1/\eta_d$, η_r and P_{aux}, while E^+, E^- and T only depend on each car and the cycle. That means the three parameters can be fitted by linear least squares. I used `scipy.optimize.lsq_linear` (Virtanen et al., 2020) with bounds $1/\eta_d \geq 1$, $0 \leq \eta_r \leq 1$ and $P_{aux} \geq 0$ so the fit cannot produce something physically impossible, and one set of parameters covers both cycles. Fitted to all 712 vehicles, $\eta_d = 0.796$ and $\eta_r = 0.648$. Both make sense: a charger, battery and drivetrain chain of roughly $0.9 \times 0.95 \times 0.9$ gives 0.77. P_{aux} went to its lower bound of 0 W, which I come back to in Section 6.1.

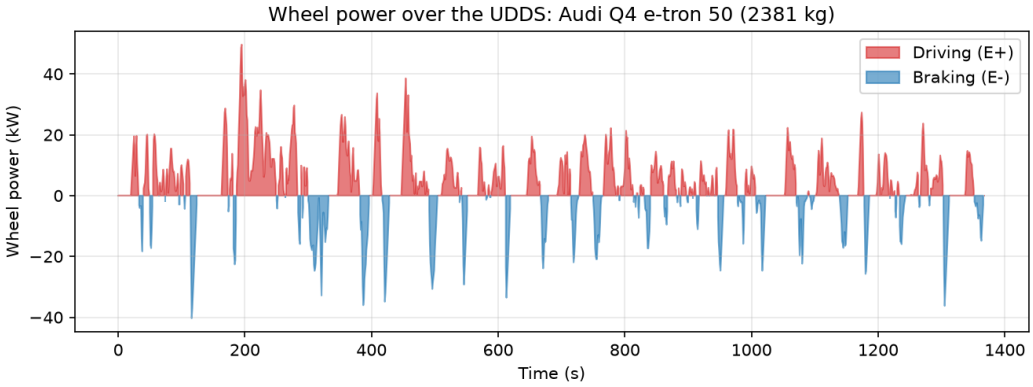


Figure 1. Wheel power over the UDDS for the 2,381 kg Audi Q4 e-tron 50. Driving energy E^+ is shaded red and braking energy E^- blue.

3.2 Numerical methods

The cycle only gives speed, so acceleration has to be calculated. I used second-order central differences for the interior points and first-order one-sided differences at the two ends (Week 7.1):

$$a_i \approx \frac{v_{i+1} - v_{i-1}}{2\,\Delta t} \tag{3}$$

E^+, E^- and D are integrated with the composite trapezoidal rule (Week 6). I also implemented Simpson's rule, which needed a small fix: both cycles have an odd number of intervals (1,369 and 765), so the 1/3 rule covers everything except the last three intervals and the 3/8 rule finishes them off. Simpson's rule turned out to be no more accurate than the trapezoidal rule here. Splitting the power into $\max(P_w, 0)$ puts a kink in the integrand every time the car switches between driving and braking, and Simpson's higher-order accuracy relies on a smooth function. I kept the trapezoidal rule because it is simpler and just as accurate (Section 4.1).

3.3 Machine-learning and hybrid models

I trained four model families from Week 5 separately for each cycle, on the inputs in Table 1. Each one sits in a scikit-learn pipeline (Pedregosa et al., 2011) that standardises the numeric inputs and one-hot encodes the categories, so the scaling is learned from training data only. The models were linear regression, a decision tree (depth and minimum leaf size tuned), SVR with an RBF kernel (C, ε and γ tuned) and a multilayer-perceptron neural network (ReLU activation, Adam optimiser, early stopping, standardised target, with layer sizes and the L2 penalty α tuned). A mean baseline, which always predicts the training average, sets the floor that any useful model has to beat.

I also used each family as a hybrid. The physics model makes the first prediction, and the ML model is trained on what is left over, the residual. The idea is that physics handles the energy balance we already understand, and the ML model picks up what it misses, such as one car's drivetrain being better than average. Hyperparameters were chosen by grid search with grouped 5-fold cross-validation on the training set (Section 4.2). For the physics + SVR model that ended up being selected, they were C = 100, ε = 1.0 kWh/100 mi and γ = 0.3 for the city cycle, and C = 100, ε = 0.5 and γ = 0.3 for the highway.

3.4 Rationale, assumptions and implementation

I made physics the main baseline because it is interpretable, its three parameters can be sanity-checked, and I expected it to behave sensibly on cars unlike the ones it was fitted to. The Week 5 models give a spread from a simple linear model to a flexible neural network. The hybrids test an idea that comes up repeatedly in the engineering-AI literature, which is to combine machine learning with physical knowledge rather than replace it (Aviles et al., 2026). The physics model assumes a flat road and a driver who follows the trace exactly (real test drivers are allowed about ±2 mph), a mass equal to the test mass with rotating inertia soaked up by the fitted parameters, efficiencies that are the same for every car and every speed, a constant accessory load, and no battery temperature or state-of-charge effects.

Everything is written as a Python package, `ev_energy`, with six modules covering data cleaning, the physics model, splitting, the ML models, validation and performance, built on NumPy, SciPy, pandas and scikit-learn. Each stage has its own script, and `scripts/run_all.py` rebuilds every result from the raw data in about a minute. The repository is on GitHub with one branch and pull request per stage, 154 pytest tests, and HTML documentation generated with pdoc.

4. Validation method and results

The model that won cross-validation did not hold up on unseen vehicles. I designed the validation so that a result like this would show up rather than get hidden, and I kept two questions separate: verification (is the code solving the equations correctly?) and validation (does the model actually predict real cars?).

4.1 Verification of the implementation

The 154 automated tests cover 97% of the code, and the test and coverage reports are in `reports/`. I checked the numerical code four ways. Against analytical solutions, central differences are exact for quadratics, Simpson's rule with the 3/8 fix is exact for cubics whether the interval count is odd or even, and the measured convergence orders come out at 2 for the trapezoidal rule and 4 for Simpson's rule, as theory says they should. Against published benchmarks, integrating each speed trace gives EPA's stated 7.45 mi and 10.26 mi. With a manufactured solution, I generated fake data from known values ($\eta_d = 0.8$, $\eta_r = 0.6$, $P_{aux} = 400$ W) and the fit recovers all three exactly. And for conservation of energy, a car with no road load that accelerates to 20 m/s and brakes back to rest gives $E^+ = E^- = \tfrac{1}{2}mv^2$ to within 0.1%.

To measure numerical error, I refined the speed trace from 1 s down to 1/16 s using linear and PCHIP interpolation (Fritsch and Carlson, 1980). I chose PCHIP because, unlike a normal cubic spline, it cannot overshoot into negative speeds. At EPA's 1 s step, consumption differs from the finest step by 1.3% on the city cycle and 0.35% on the highway, converging at roughly first order with linear interpolation and close to second order with PCHIP. That is small next to a model error of around 12%. Stability is not really a concern here, because nothing is stepped forward in time; the most error-sensitive part is the finite-difference derivative, which amplifies any noise in the speed trace. I also wrote a leakage test. It adds 100 kWh/100 mi to the targets in one cross-validation fold and

checks that the predictions for that fold do not move. It passes for every model, which shows that no scaler, model or physics fit inside the hybrids ever sees held-out data.

4.2 Validation design

A lot of car models appear in several model years with identical specifications. A random row-by-row split would therefore test partly on copies of training cars and make every model look better than it is. I grouped every split by vehicle model instead. I held out 20% of the 263 models at random (seed 42), giving a test set of 175 vehicles from 53 models and a training set of 537 vehicles from 210 models. The two sets look alike, with mean city consumption of 25.5 and 25.1 kWh/100 mi. I tuned hyperparameters with grouped 5-fold cross-validation on the training set only. I had to assign the folds myself with a seeded shuffle, because scikit-learn's `GroupKFold` gave different folds on two computers with different library versions. I then repeated the cross-validation over five different fold assignments, since a single run could not separate the top models.

RMSE in kWh/100 mi is the main metric, because it punishes large errors and large errors are what matter in engineering use. I also report MAE, R^2 and maximum error. To count as useful, a model had to beat the mean baseline, and a more complex model had to beat physics alone to justify the extra complexity. Cross-validation did influence which model I selected. The test set did not. Before opening it, I fixed the rule: one model for both cycles, the one with the lowest average of its city and highway mean CV RMSE. I opened the test set once, after that choice was made, and did not change it afterwards.

4.3 Cross-validation and test results

Table 3. Repeated cross-validation (mean ± standard deviation over five grouped 5-fold runs) and held-out test RMSE with 95% intervals from 2,000 bootstrap resamples of whole vehicle models. RMSE in kWh/100 mi; all ten models are in Appendix Tables A1 and A2.

Model	City CV	Highway CV	City test [95% CI]	Highway test [95% CI]
Mean baseline	5.33 ± 0.03	4.42 ± 0.00	5.46	4.70
Physics only	3.39 ± 0.01	2.43 ± 0.00	3.62 [2.75, 4.50]	2.39 [1.58, 3.18]
Linear	3.27 ± 0.07	2.36 ± 0.05	3.18 [2.27, 3.93]	2.04 [1.38, 2.72]
Neural network	3.00 ± 0.10	2.43 ± 0.14	3.22 [2.31, 3.95]	2.16 [1.43, 2.89]
Physics + MLP	2.97 ± 0.17	2.46 ± 0.22	3.06 [2.20, 3.78]	2.03 [1.35, 2.74]
Physics + SVR (selected)	**2.88 ± 0.10**	**2.36 ± 0.09**	**3.82 [2.03, 5.19]**	**2.80 [1.36, 3.98]**

In cross-validation, physics + SVR was clearly ahead on the city cycle, at 2.88 against 3.39 for physics, about 15% lower. On the highway the top four models were all within their run-to-run spread of each other. Physics was easily the most stable model, with a standard deviation of 0.01 or less, and the selection rule picked physics + SVR with an average RMSE of 2.62.

Then the test set told a different story. The selected model did worse than physics alone on both cycles, and it beat physics in only 44% (city) and 30% (highway) of bootstrap resamples. With only 53 test models the intervals are wide, and no model is clearly better than physics. The closest is physics + MLP, which beats it in 95% of resamples, but the interval for the difference only just reaches zero. Part of the gap between the CV estimate (2.88) and the test result (3.82) is the usual optimism from choosing hyperparameters on the same data used to score them.

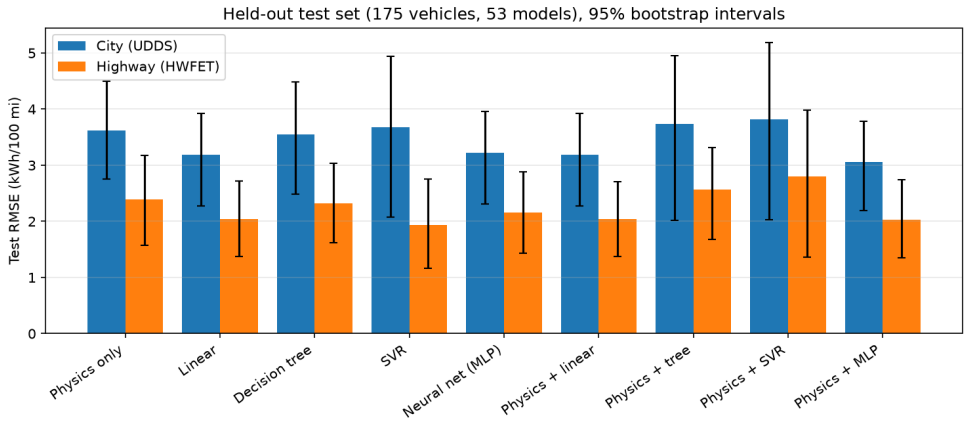


Figure 2. Test RMSE for each model with 95% bootstrap intervals.

4.4 Failure analysis and unsupported inputs

The single worst test error was the 2025 Maserati Grecale Folgore, whose road-load inputs sit right at the edge of the training data (f_0 = 285 N against a data maximum of 295 N, and f_1 = 7.2 N/(m/s), in the top few percent). Its measured city consumption was 26.9 kWh/100 mi. Physics predicted 31.7, which is too high but sensible. Physics + SVR predicted 51.6. The RBF kernel with γ = 0.3 and C = 100 had learned sharp local patterns, and once it was outside the region it was trained on, its predictions ran away. The hybrid's typical error was actually lower than physics (city MAE 2.20 against 2.79 kWh/100 mi). Its RMSE was worse because of a few very large misses, and for an engineering tool that is the more dangerous way to be wrong.

To catch these cases I added a domain check that flags any test vehicle with a numeric input outside the 1st to 99th percentile of the training data. It flagged 4 models (15 vehicles): the 2022 Ford F-150 Lightning, the 2023 Nissan Ariya Engage, the 2023 Rivian R1S 20in and the Maserati. On those vehicles the physics + tree hybrid's city RMSE jumped from 3.29 to 6.89, while physics alone stayed at 2.89. Any hybrid that was actually used should sit behind a check like this and fall back to physics when the inputs look unusual.

4.5 Temporal check and sensitivity analysis

To see how the models would cope with next year's cars, I trained them on 2022 to 2024 vehicles (505) and tested on 2025 to 2026 models they had never seen (60 vehicles, 41 models), keeping the Stage 3 hyperparameters. Physics was the best model on both cycles, with an RMSE of 2.05 kWh/100 mi in the city (R^2 0.62) and 1.18 on the highway (R^2 0.84). Physics + SVR scored R^2 values of −0.58 and −0.53, which is worse than just predicting the average.

For the sensitivity analysis I changed each test vehicle's mass by ±10%, then separately its aerodynamic coefficient f_2, and expressed the response as an elasticity: the percentage change in predicted consumption for a 1% change in the input (Figure 3). Physics moved in the right direction for every vehicle, and the pattern makes sense. A 1% increase in mass raises consumption by 0.43% in the city but only 0.12% on the highway, while a 1% increase in drag raises it by 0.19% and 0.46%, because mass matters most in stop-start driving and drag matters most at speed. Linear regression was also right every time. Every flexible model, though, predicted lower consumption for a heavier or draggier car for some vehicles: the neural network for 17 to 34% of test vehicles, and physics + SVR for 5 to 33%, worst when mass was increased on the highway. Even where their accuracy looked good, these models had not learned the physics.

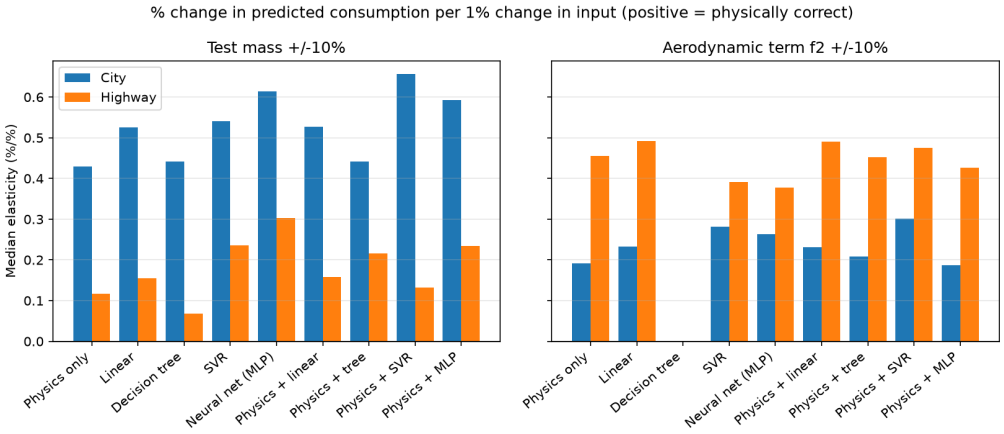


Figure 3. Median elasticity of predicted consumption to mass and to the aerodynamic coefficient f_2, by model and cycle. Positive values are physically correct.

4.6 Interpretation: four well-known EVs

To connect the predictions to numbers people actually see, I converted predicted consumption into approximate window-sticker MPGe (label MPGe ≈ 0.7 × 3,370.5 / c, combining the cycles as 55% city and 45% highway) and compared it with the official figures I looked up on fueleconomy.gov (US Department of Energy and US Environmental Protection Agency, n.d.). All four cars are in the test set.

Table 4. Combined label MPGe for four well-known 2024 EVs.

2024 vehicle	Measured	Physics	Physics + SVR	Official label
Tesla Model 3 RWD	132.5	120.9	129.9	132
Tesla Model Y Long Range AWD	111.6	103.6	103.3	117
Hyundai Ioniq 6 Long Range RWD (18 in)	137.2	121.2	133.2	140
Kia EV6 Standard Range RWD	118.9	111.1	121.2	117

The measured values convert to within 5% of the official labels, which supports using the 0.7 factor. For these mainstream cars the hybrid lands close to the official figures, while physics underestimates their efficiency by 5 to 13%, because one average drivetrain efficiency cannot represent drivetrains that are better than average. Using published battery capacities (77.4 kWh for the Ioniq 6, Green Car Reports, n.d.; 58.0 kWh for the EV6, Kia America, n.d.) and an assumed charging efficiency of 85 to 95%, the hybrid predicts a label range of 322 to 360 miles for the Ioniq 6 (EPA says 361) and 220 to 245 miles for the EV6 (EPA says 232). Tesla does not publish battery capacities, so I did not estimate range for the Teslas.

4.7 What the validation cannot establish

It only covers laboratory cycles, not heating, cooling, temperature or driving style. With 53 test models the 95% intervals overlap for every model, so it cannot show that any model is definitely better than physics. My recommendation to prefer physics (Section 6.2) was made after seeing the test results, so its test score is no longer an unbiased estimate of how it will do in future. Finally, vehicles well outside the training range, such as heavy commercial EVs, get flagged by the domain check but cannot be predicted reliably.

5. Comparison and optimisation

Weighing accuracy against robustness, cost and interpretability, physics comes out as the best overall choice, and I made its core calculation 104 times faster than the first version. All timings are from the target machine (Apple Silicon, 12 cores, Python 3.14.8, NumPy 2.5.3) unless I say otherwise.

5.1 Comparison of the alternatives

Table 5. Comparison of physics, the selected hybrid and the neural network across the brief's criteria.

Criterion	Physics only	Physics + SVR	Neural network
Test RMSE, city / highway (kWh/100 mi)	3.62 / 2.39	3.82 / 2.80	3.22 / 2.16
Robustness: CV spread (city)	±0.01	±0.10	±0.10
Robustness: new 2025 to 2026 models, highway R^2	0.84	−0.53	0.63
Physically correct sensitivity	every vehicle	wrong for 5 to 33%	wrong for 17 to 34%
Training time, both cycles	0.7 ms	17 ms	181 ms
Fitted parameters	3, each physical	3 + support vectors	about 900 weights
Interpretability	each parameter checkable	partial	low
Behaviour outside training range	graceful	can fail badly	uncertain

The neural network has the lowest test RMSE of the three, but its interval overlaps with physics, it responds the wrong way to mass and drag for some cars, and it takes about 260 times longer to train. Physics is the only one that is robust, interpretable and physically consistent.

5.2 Baseline and improved physics kernel

The kernel is the part that computes wheel power for every vehicle at every time step and integrates it. I implemented it four ways, and all four give the same answers to within 3×10^{-15} (round-off), which the tests check. The times in Table 6 are the fastest of 7 runs on the UDDS (3 runs for the slow baseline), measured with Python's `time.perf_counter`, with peak memory from `tracemalloc`.

Table 6. Run time and peak memory of the four kernel implementations.

Implementation	712 vehicles	7,120 vehicles	Peak memory at 7,120
Pure Python loops (baseline)	0.237 s	not run	not run
NumPy, loop over vehicles	0.0046 s	0.045 s	0.6 MB
NumPy, fully vectorised	0.0023 s	0.024 s	234 MB
NumPy, chunks of 712 (selected)	**0.0023 s**	**0.023 s**	**23.7 MB**

Vectorising with NumPy (Harris et al., 2020) gets rid of the Python interpreter's overhead on every single operation, which makes the kernel 104 times faster than the baseline (Figure 4).

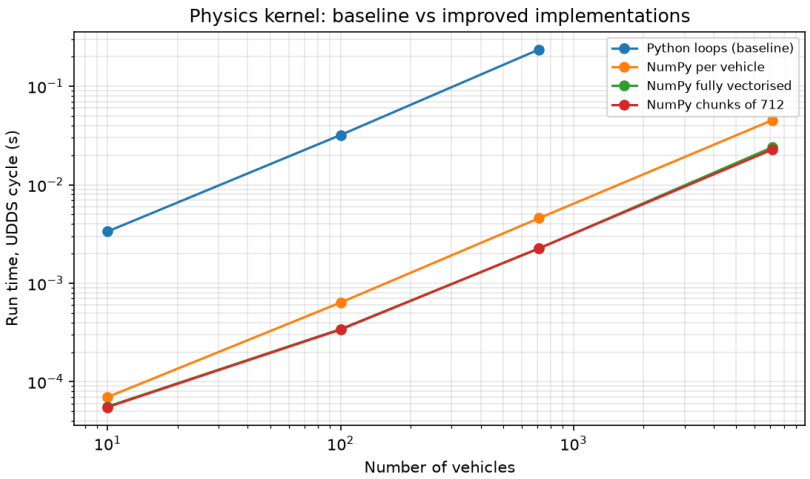


Figure 4. Run time against number of vehicles for the four kernel implementations (log-log).

5.3 FLOPs and arithmetic intensity

For each vehicle at each time step, the kernel does 7 FLOPs to get the wheel power ($m \cdot a$, $f_1 \cdot v$, $f_2 \cdot v^2$, three additions and the multiplication by v), 1 negation for the braking part and 2 additions for the integrals, which is 10 FLOPs in total. The v^2 term only depends on time, so it is computed once per step and shared across every vehicle rather

than counted per vehicle. One UDDS pass over 712 vehicles is therefore 712 × 1,370 × 10 ≈ 9.75 million FLOPs. In NumPy, every operation writes a full-size temporary array that the next operation reads back in. Counting 10 writes and 11 reads of 8-byte numbers gives 168 bytes moved per vehicle per time step, so the arithmetic intensity (AI) is

$$\mathrm{AI}_{\mathrm{NumPy}} = \frac{10\ \mathrm{FLOP}}{168\ \mathrm{B}} \approx 0.06\ \mathrm{FLOP/B}, \qquad \mathrm{AI}_{\mathrm{fused}} = \frac{10\ \mathrm{FLOP}}{16\ \mathrm{B}} \approx 0.63\ \mathrm{FLOP/B} \tag{4}$$

where the fused figure assumes an ideal compiled loop that only streams in v and a. Both are well below a modern CPU's machine balance of several FLOP per byte, so the kernel is memory-bandwidth bound. It runs at about 4.3 GFLOP/s, a small fraction of what the processor can do, which fits. The speed is limited by moving data around, not by the arithmetic.

5.4 Chunk size: a hardware-dependent optimisation

Since the kernel is limited by memory, I also tried processing the vehicles in chunks so that each temporary array is smaller. On Claude's test machine, an Intel Xeon with a 1 MB L2 cache, chunks of 32 vehicles (about 350 kB per array) were fastest: 1.8 times faster than doing all 712 at once, and 2.4 times faster at 7,120 vehicles, because the full 7.8 MB arrays no longer fit in cache. On my Apple Silicon machine, which has a much bigger L2 cache, the full arrays still fit, so doing everything at once was fastest and chunks of 32 were about 20% slower. I settled on chunks of 712 vehicles. On my machine that is just as fast as full vectorisation (0.0023 s), but it also stops memory from growing with the dataset: at 7,120 vehicles, peak memory is 23.7 MB instead of 234 MB. I kept the Xeon results in the repository for reference.

5.5 Time step, profiling and parallel tuning

Halving the time step roughly doubles the run time, but at 1 s the numerical error is already only 1.3% (city) and 0.35% (highway) against a model error of about 12%. A finer step would cost time without improving anything that matters, so I kept 1 s.

Profiling one full cross-validation run with `cProfile` (Python Software Foundation, n.d.), which took 3.9 s, showed that neural-network training accounts for about 80% of the time (3.16 s). About a third of that is the network scoring its early-stopping validation set after every epoch. The physics model barely registers. Running the neural network's grid search across all 12 cores instead of one gave about a 2 times speed-up (1.9 to 2.2 times over two runs) and chose the same hyperparameters. It is nowhere near 12 times because each fit only takes about 0.1 s, so most of the time goes on starting worker processes and passing data to them. That is Amdahl's law in practice.

5.6 Final design

The final design uses the physics model as the main predictor, computed with the vectorised kernel in chunks of 712 vehicles at EPA's 1 s time step with the trapezoidal rule. I would only use a hybrid for mainstream vehicles inside the training range, behind the domain check from Section 4.4, and fall back to physics whenever it flags an input.

6. Limitations and lessons learned

6.1 Limitations

- The best test error, about 3 kWh/100 mi on the city cycle, is roughly 20 times the 0.16 noise floor. Most of what is left comes from information the data simply does not contain, such as motor type, tyres, battery chemistry and software calibration, rather than from the methods.
- Using one drivetrain efficiency for every car and speed is why physics underestimates the efficiency of the best cars by 5 to 13%. It is probably also why P_{aux} hit zero: without the bound, the fit wanted about −555 W, which is impossible and tells me the model is compensating for something it cannot represent.
- With 53 test models the 95% intervals overlap for every model, and the data-driven models are unreliable outside the training range. The domain check can spot those inputs but cannot fix them.
- The whole-number rule also threw out about 10 vehicles that are probably genuine, including the 2024 to 2026 Tesla Model 3 Long Range RWD, which sits within 15% of physics. I accepted losing 1.4% of the data to keep a rule that does not depend on the model.

- The range estimates rely on nominal battery capacities, an assumed charging efficiency of 85 to 95%, and EPA's approximate 0.7 label factor.

6.2 Recommendation and proposed improvements

For early-stage design estimates, I recommend the physics model. It held up best across folds, model years and vehicle types, and it is by far the cheapest to run. The hybrids help for mainstream cars, but only behind the domain check. Three improvements follow directly from the results:

1. Make the efficiency depend on speed or load instead of being a single constant. That should fix both the negative accessory-power result and the underestimate for efficient cars, while keeping the model physical.
2. Add more inputs, such as battery capacity and motor type from fueleconomy.gov. This targets the largest source of remaining error. I decided against it for this project because matching vehicle names across the two sources would have taken too long.
3. Constrain the hybrids so that predicted consumption can only go up with mass and drag. That would remove the physically wrong responses found in Section 4.5.

6.3 Lessons learned

The biggest lesson was that cross-validation can mislead, and that the test set has to stay closed until the end. The model that won cross-validation lost on unseen data. One CV run suggested a city RMSE of 2.63 for physics + SVR, five repeated runs said 2.88, and the test said 3.82. Because I had fixed the selection rule before opening the test set, the failure was out in the open. If I had switched to whichever model won on the test set, its score would have been just as optimistic as the CV score was.

I also learned that a physics model doubles as a data check. The physics cross-check exposed the window-sticker values, which a purely statistical model would have happily learned to reproduce. The problems that worried me most were the ones that raised no error at all: the mixed units, the window-sticker values, folds that changed between library versions, and a file update that silently failed to copy. Each one was caught because I had a known value to compare against, whether a test count, a benchmark or a physics prediction.

Finally, optimisation depends on the hardware. The chunk size that was fastest on one machine was slower on mine, and measuring on my own computer changed the final design. Coming into this project I had never used Git, GitHub, virtual environments or automated testing. Working with branches, pull requests, tests and generated documentation showed me how engineering software is kept reproducible, and it is a way of working I can take into student-team projects and industry.

7. AI-use reflection

AI wrote most of the code and helped refine this report to a professional engineering standard. The checking, the comparisons against outside sources and the engineering decisions were mine. Callum allowed unrestricted use of AI for this task, and I kept a session-by-session record in `learning_log.md` in the repository.

The tool I used was Claude (Anthropic – Model Opus 5.5), through the claude.ai chat interface and its code-execution environment. It contributed in setting up the project, the structure and the codes. It reviewed the brief with me and assessed possible datasets against the rubric, inspected the raw EPA data, and worked through the physics derivation with me. It wrote essentially all of the Python code, the 154 tests, the figures and some parts of the README. It also gave me recommended structures to organise the project and walked me through the testing step by step. I then drafted this report from my results and learning log, which it then reviewed and edited for me.

I checked its output in several ways. I ran every script and test on my own machine and compared the output line by line with Claude's runs on its environment. I agreed and approved the physics derivation step by step before any code was written. I checked results against independent sources: EPA's published cycle distances, and official label figures that I looked up myself on fueleconomy.gov for multiple vehicles. The automated tests also check the code against analytical solutions, a manufactured solution and conservation of energy.

Even though Claude's suggestions influenced my decisions, many of the engineering decisions and final recommendations were mine alone.

The AI also got things wrong, and these turned out to be some of the most useful moments:

- It first counted 11 FLOPs per sample, then corrected this to 10 after noticing that v^2 is computed once per time step rather than per vehicle.
- It recommended chunks of 32 based on its own Intel Xeon. On my Apple Silicon machine that was about 20% slower, so my measurements overturned it.
- scikit-learn's `GroupKFold` gave different results on my machine and Claude's. My output not matching is what exposed it, and it led to explicit fold assignment and repeated cross-validation.

Looking back, AI massively increased what I could build in the time given, and the expanded the scope hugely. The risk was that I would end up understanding less of what was built. I dealt with that by agreeing each design decision before it was implemented, checking every stage myself and keeping the learning log, and whenever the AI was wrong or my results did not match its own, is when I learnt the most because I asked questions back to figure out what was actually happening. Chasing those down taught me more about cross-validation, reproducibility and hardware performance than the parts that worked first time.

References

Aviles, M., Aceves-Fernández, M. A., Odry, A., Álvarez-Alvarado, J. M. and Rodriguez-Resendiz, J. (2026). Artificial intelligence for engineering solutions: trends, challenges, and future directions. *Eng*, 7, 317. https://doi.org/10.3390/eng7070317

Fritsch, F. N. and Carlson, R. E. (1980). Monotone piecewise cubic interpolation. *SIAM Journal on Numerical Analysis*, 17(2), 238–246.

Green Car Reports (n.d.). 2023 Hyundai Ioniq 6 rated at 361 miles, tops Model 3 Long Range. https://www.greencarreports.com/news/1138626_2023-hyundai-ioniq-6-rated-at-361-miles-tops-model-3-long-range (accessed October 2026; 77.4 kWh pack and 361-mile EPA range for the SE Long Range RWD, unchanged for 2024).

Harris, C. R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.

Kia America (n.d.). Kia announces 2024 EV6 pricing. Kia Media. https://kiamedia.com/us/en/models/ev6/2024 (accessed October 2026).

Pedregosa, F. et al. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

pdoc (n.d.). pdoc: API documentation for Python projects. https://pdoc.dev

Python Software Foundation (n.d.). The Python profilers; tracemalloc: trace memory allocations. Python 3 documentation. https://docs.python.org/3/library/profile.html

US Department of Energy and US Environmental Protection Agency (n.d.). fueleconomy.gov. https://www.fueleconomy.gov (accessed October 2026).

US Environmental Protection Agency (n.d.-a). Data on cars used for testing fuel economy. https://www.epa.gov/compliance-and-fuel-economy-data/data-cars-used-testing-fuel-economy (model years 2022 to 2026, downloaded October 2026).

US Environmental Protection Agency (n.d.-b). Dynamometer drive schedules. https://epa.gov/vehicle-and-fuel-emissions-testing/dynamometer-drive-schedules (accessed October 2026).

Virtanen, P. et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272.

Appendix: full results tables

Table A1. Repeated grouped cross-validation on the training set: RMSE (kWh/100 mi), mean ± standard deviation over five fold assignments, sorted by the selection criterion.

Model	City RMSE	Highway RMSE	Average
Physics + SVR	2.880 ± 0.104	2.364 ± 0.093	2.622
Physics + MLP	2.972 ± 0.165	2.462 ± 0.219	2.717
Neural network (MLP)	3.002 ± 0.102	2.432 ± 0.141	2.717
Physics + linear	3.259 ± 0.069	2.360 ± 0.047	2.810
Linear	3.266 ± 0.069	2.363 ± 0.047	2.815
Physics + tree	3.177 ± 0.309	2.501 ± 0.093	2.839
SVR	3.258 ± 0.065	2.511 ± 0.041	2.884
Physics only	3.391 ± 0.014	2.425 ± 0.004	2.908
Decision tree	3.610 ± 0.125	3.269 ± 0.089	3.439
Mean baseline	5.331 ± 0.027	4.421 ± 0.003	4.876

Table A2. Held-out test set (175 vehicles, 53 models). RMSE and MAE in kWh/100 mi; 95% intervals from 2,000 vehicle-model bootstrap resamples.

Model	City RMSE [95% CI]	Hwy RMSE [95% CI]	City MAE	Hwy MAE	City R^2	Hwy R^2
Mean baseline	5.46	4.70	4.56	3.66	−0.01	−0.01
Physics only	3.62 [2.75, 4.50]	2.39 [1.58, 3.18]	2.79	1.61	0.56	0.74
Linear	3.18 [2.27, 3.93]	2.04 [1.38, 2.72]	2.42	1.49	0.66	0.81
Decision tree	3.55 [2.48, 4.49]	2.32 [1.62, 3.03]	2.63	1.63	0.58	0.76
SVR	3.68 [2.08, 4.94]	1.93 [1.16, 2.75]	2.16	1.31	0.54	0.83
Neural network (MLP)	3.22 [2.31, 3.95]	2.16 [1.43, 2.89]	2.23	1.50	0.65	0.79
Physics + linear	3.18 [2.27, 3.93]	2.04 [1.38, 2.71]	2.42	1.49	0.66	0.81
Physics + tree	3.74 [2.02, 4.96]	2.56 [1.68, 3.32]	2.02	1.73	0.53	0.70
Physics + SVR (selected)	**3.82 [2.03, 5.19]**	**2.80 [1.36, 3.98]**	**2.20**	**1.37**	**0.51**	**0.64**
Physics + MLP	3.06 [2.20, 3.78]	2.03 [1.35, 2.74]	2.09	1.45	0.69	0.81

Table A3. Temporal check: trained on 2022 to 2024 vehicles (505), tested on 41 new 2025 to 2026 models (60 vehicles). RMSE in kWh/100 mi.

Model	City RMSE	Highway RMSE	City R^2	Highway R^2
Mean baseline	3.37	3.09	−0.03	−0.13
Physics only	**2.05**	**1.18**	**0.62**	**0.84**
Linear	2.08	1.44	0.61	0.76
Decision tree	2.62	1.84	0.38	0.60
SVR	3.96	1.31	−0.42	0.80
Neural network (MLP)	2.52	1.76	0.43	0.63
Physics + linear	2.08	1.44	0.61	0.76
Physics + tree	3.44	1.42	−0.07	0.76
Physics + SVR	4.18	3.59	−0.58	−0.53
Physics + MLP	2.29	1.40	0.53	0.77

Decision-tree results differed by up to 0.10 kWh/100 mi between the two machines used (scikit-learn 1.8 and 1.9) because of tie-breaking changes; all values are from the target machine.

Table A4. Data cleaning steps.

Step	Rows remaining
Raw rows, all fuels and procedures	21,479
EV charge-depleting city and highway tests	1,606
Valid efficiency values (placeholders such as 999 removed)	1,566
Complete, plausible inputs (placeholder power below 50 hp removed)	1,546
After averaging repeated tests of the same vehicle	1,494
Vehicles with both a city and a highway result	741
Passing physical plausibility checks	740
After excluding suspected window-sticker values	712 (263 vehicle models)